

Big data: architectures and data analytics Recap

Simone Colagiovanni

Politecnico di Torino a.a 2024/2025

Abstract

This paper is structured as follows: the first section provides a summary of the most important theoretical concepts related to MapReduce and Spark, covering topics up to Spark SQL. The second section offers insights and tips derived from observations during the lab sessions and exercises. Finally, the paper concludes with a practical guide outlining common issues that may arise during the exam and strategies to address them effectively.

Contents

1	Theory Map Reduce	4
1.1	Input Formats	4
1.1.1	TextInputFormat	4
1.1.2	SequenceFileInputFormat	4
1.1.3	KeyValueTextInputFormat	4
1.2	OutputFormat	4
1.3	Parameters/Arguments of the Application	5
1.4	Mapper	5
1.5	Reducer	6
1.6	Personalized Data Types and Values	6
1.7	Combiner	7
1.8	Shuffle and Sort	8
1.9	Counters	8
1.9.1	Example of User-Defined Counters	8
1.9.2	User-Defined Counters	9
1.10	Summarization Patterns	10
1.10.1	Numerical Summarizations	10
1.10.2	Inverted Index Summarizations	10
1.10.3	Counting with Counters	10
1.11	Filtering Patterns	10
1.11.1	Filtering	10
1.11.2	Top K	11
1.11.3	Distinct	11
1.11.4	Multiple Inputs	12
1.11.5	Multiple Outputs	12
1.11.6	Driver Implementation	13
1.11.7	Map-Only Example	13
1.12	Data Organization Patterns	14
1.12.1	Binning	14
1.12.2	Shuffling	14
1.13	Metapatterns	14

1.13.1	Job Chaining	14
1.14	Join Patterns	15
1.14.1	Reduce-Side Natural Join	15
1.14.2	Map-Side Natural Join	16
2	Theory Spark	17
2.1	RDD Creation	17
2.1.1	Create RDDs from Files	17
2.1.2	Create RDDs from a Local Python Collection	17
2.1.3	Save RDDs	18
2.1.4	Retrieve the Content of RDDs	18
2.2	Transformations and Actions	18
2.2.1	Example of a Solution Based on <code>def</code>	18
2.3	Basic transformations based on one single RDD: Summary	20
2.4	Basic transformations based on two RDDs: Summary	20
2.5	Basic actions: Summary	21
2.6	Transformations on RDDs of key-value pairs: Summary	22
2.7	Transformations on two RDDs of key-value pairs	23
2.8	Actions on RDDs of key-value pairs: Summary	23
2.9	RDDs of numbers: Summary	23
2.10	Persistence and Cache	23
2.11	Accumulators	24
2.12	Personalized Accumulators	25
2.13	Broadcast Variables	25
3	Additional Information for MapReduce	27
3.1	Using Placeholders in the Mapper Phase for Counting Occurrences	27
3.2	Using Custom Data Types to Preserve Key-Value Relationships in Map and Reduce Phases	27
3.3	Types for Mapper, Driver and Reducer	27
3.4	Guide: When to Use <code>setup</code> and <code>cleanup</code> with Custom Data Types and Data Structures in Hadoop	28
3.5	Useful pattern and function - Mapper	33
3.5.1	Splitting the sentence in a <code>TextInputFile</code>	33
3.5.2	Splitting each record of a csv file by using the field separator	33
3.5.3	Extracting the value in a <code>KeyValueTextInputFormat</code>	34
3.5.4	Emitting pair in a <code>TextInputFile</code>	34
3.6	Useful pattern and function - Reducer	34
3.6.1	Operation on the set of values	34
4	Additional Information for Spark	36
4.1	Useful pattern and function	36
4.1.1	Extracting a Specific Field from an RDD	36
4.1.2	<code>takeOrdered()</code>	36
4.1.3	<code>list()</code>	36
4.1.4	Retrieving the Maximum Value from <code>ValuesRDD</code>	37
4.1.5	Retrieving Lines Associated with the Maximum Value	37
4.1.6	Associating a Number to Each Value in an RDD	38

4.1.7	Computing the Sum and the Avg	38
4.1.8	Creating an RDD with Paired Fields from a CSV File	38
4.1.9	Computing the Maximum Value for Each Key	38
5	Closed Question	40
5.1	Similar behaviour in Spark Streaming	40
5.2	General Strategy for Solving Spark Streaming Window-Based Exercises	41
5.3	Practical Steps to Solve Spark RDD Read Count Problems	42
5.4	Determine the number of mapper instantiated	43
5.5	Computation of <code>reduce</code> method invocations	44
5.6	Computation of <code>map</code> method invocations	44
6	Hadoop Exam	46
6.1	Deciding the Number of Jobs in a Hadoop Application	46
6.2	Some basic instruction for mapper and reducer	47
6.3	Attribute with the Largest Number of Items	47
7	Spark Exam	54
7.1	<code>keys()</code> Function	54
7.2	<code>values()</code> Function	54
7.3	Subtracting Elements in a Pair RDD	54
7.3.1	Subtracting Based on Keys	54
7.3.2	Subtracting Based on Values	54
7.3.3	Key-Value Pair Subtraction (Exact Matches)	55
7.4	Usage of <code>reduceByKey</code>	55
7.5	When to Use <code>reduceByKey</code> and <code>reduce</code>	57
7.6	Using the <code>mapValues</code> Function	58
7.7	Using the <code>subtractByKey</code> Function	59
7.8	Accessing Fields in Nested Tuples - Single pair	59
7.9	Accessing Fields in Nested Tuples - Double pair	60
7.10	How the <code>join</code> works in PySpark	61
7.11	Types of Joins in Spark	63
7.12	Comparing Total Statistics Across Two Different Years	64
7.13	Computation of Max, Min, Total, and Count for a Specific Field	67
7.14	Union Function for RDD and Pair RDD in Apache Spark	68

1 Theory Map Reduce

1.1 Input Formats

1.1.1 TextInputFormat

Files are broken into lines either linefeed or carriage-return are used to signal end of line. One pair (key, value) is emitted for each line of the file:

- **Key** is the position (offset) of the line in the file
- **Value** is the content of the line

In this case, in the mapper phase, the key type will be a `LongWritable` and the value type will be `Text`.

1.1.2 SequenceFileInputFormat

An InputFormat for sequential/binary files.

```
// Set path of the input file/folder (if it is a folder, the
    job reads all the files in the specified folder) for this
job
FileInputFormat.addInputPath(job, inputPath);
```

1.1.3 KeyValueTextInputFormat

Each line of the file (that is the result of a previous elaboration) must have the format `key<separator>value`. The default separator is `\t` (tab). Files are broken into lines either linefeed or carriage-return are used to signal end of line. Each line is split into key and value parts by considering the separator symbol/character. One pair (key, value) is emitted for each line of the file:

- **Key** is the text preceding the separator
- **Value** is the text following the separator

When using `KeyValueTextInputFormat` in Hadoop, both the input `key` type and `value` type for the Mapper are always `Text`.

1.2 OutputFormat

`TextOutputFormat` is an OutputFormat for plain text files. For each output (key, value) pair, `TextOutputFormat` writes one line in the output file. The format of each output line is `key\tvalue\n`.

`SequenceFileOutputFormat` is an OutputFormat for sequential/binary files.

1.3 Parameters/Arguments of the Application

- `args[0]`: number of instances of the reducer $\rightarrow$ `Integer.parseInt(args[0])`.
- `args[1]`: path of the input file.
- `args[2]`: path of the output folder.

1.4 Mapper

Mapper Class

```
/* Mapper Class */
class WordCountMapper extends Mapper<
LongWritable, // Input key type
Text,         // Input value type
Text,         // Output key type
IntWritable> // Output value type
```

When I have as `InputFormat TextInputFormat`, the input key will be `LongWritable` and the input value will be `Text`.

Implementation of the map method

```
/* Implementation of the map method */
protected void map(
LongWritable key, // Input key type
Text value,       // Input value type
Context context) throws IOException, InterruptedException {
    /* Insert your code here */
}
```

To emit the pair at the end, I have to use `context.write(new OutputKeyType(..), new OutputValueType())`; where `OutputKeyType` and `OutputValueType` are the same that I have declared in the mapper class.

1.5 Reducer

Reducer Class

```
/* Reducer Class */
class WordCountReducer extends Reducer<
Text,          // Input key type -> emitted as key by the
    Mapper
IntWritable,   // Input value type -> emitted as a value by
    the Mapper
Text,          // Output key type
IntWritable>  // Output value type
```

In the ReducerClass, the InputKeyType and InputValueType are the same as the pair emitted by the Mapper.

Implementation of the reduce method

```
/* Implementation of the reduce method */
protected void reduce(
Text key,          // Input key type
Iterable<IntWritable> values, // Input value type
Context context) throws IOException, InterruptedException {
    /* Insert your code here */
}
```

As before, to emit the pair at the end use `context.write(new OutputKeyType(..), new OutputValueType());` where `OutputKeyType` and `OutputValueType` are the same that I have declared in the reducer class. The input value type to the reduce method is always an `Iterable` of the value type emitted by the map method

1.6 Personalized Data Types and Values

Personalized Data Types are useful when the value of a key-value pair is a complex data type. Personalized Data Types are defined by implementing the `org.apache.hadoop.io.Writable` interface. The following methods must be implemented:

- `public void readFields(DataInput in)`
- `public void write(DataOutput out)`

To properly format the output of the job usually also the following method is “redefined”: `public String toString()`. Suppose we are interested in “complex” values composed of two parts: a counter (`int`) and a sum (`float`). An ad-hoc Data Type can be used to implement this complex data type in Hadoop.

Complex Value Data Type Example

```

/* Private variables */
private float sum = 0;
private int count = 0;
/* Methods to get and set private variables of the class */
public float getSum() {
    return sum;
}
public void setSum(float sumValue) {
    sum = sumValue;
}
public int getCount() {
    return count;
}
public void setCount(int countValue) {
    count = countValue;
}

/* Methods to serialize and deserialize the contents of the
   instances of this class */
@Override /* Serialize the fields of this object to out */
public void write(DataOutput out) throws IOException {
    out.writeFloat(sum);
    out.writeInt(count);
}
@Override /* Deserialize the fields of this object from in */
public void readFields(DataInput in) throws IOException {
    sum = in.readFloat();
    count = in.readInt();
}
/* Specify how to convert the contents of the instances of
   this class to a String
   * Useful to specify how to store/write the content of this
   class
   * in a textual file */
public String toString() {
    String formattedString =
        new String("sum=" + sum + ",count=" + count);
    return formattedString;
}

```

1.7 Combiner

A **Combiner** is a **local reducer** that performs partial aggregation of Mapper output to **reduce** the amount of data sent to the **Reducer**. It is useful for **commutative** and **associative**

operations (like sum, count, etc.) and can lead to **performance improvements** by reducing network traffic.

How the Combiner Works in This Example:

- The **Mapper** emits (word, 1) pairs.
- The **Combiner** then **locally aggregates** these (word, 1) pairs within each Mapper. For example, if a Mapper processes the word "hello" multiple times in different records, it can combine them to emit (hello, 3) instead of multiple (hello, 1) pairs.
- The **Reducer** then receives the **already partially aggregated data**. It only sums up the counts that the Combiner didn't already handle.

Combiner Example: Word Count Reducer

```
public class WordCountReducer extends Reducer<Text,
    IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();

    protected void reduce(Text key, Iterable<IntWritable>
        values, Context context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get(); // Sum up the counts for this
                               word
        }
        result.set(sum);
        context.write(key, result); // Emit the total count
                                     for the word
    }
}
```

1.8 Shuffle and Sort

This phase happens **automatically** between the mapper and the reducer in the MapReduce framework. The framework delivers to the reducer groups of pairs in the format: (key, list(values)), where list(values) contains all the values associated with a specific key.

1.9 Counters

1.9.1 Example of User-Defined Counters

In the driver:

Defining Counters

```
public static enum COUNTERS {
    ERROR_COUNT,
    MISSING_FIELDS_RECORD_COUNT
}
```

This enum defines two counters:

- `COUNTERS.ERROR_COUNT`
- `COUNTERS.MISSING_FIELDS_RECORD_COUNT`

This example increments the `COUNTERS.ERROR_COUNT` counter. In the mapper or the reducer:

Incrementing a Counter

```
context.getCounter(COUNTERS.ERROR_COUNT).increment(1);
```

This example retrieves the final value of the `COUNTERS.ERROR_COUNT` counter. In the driver:

Retrieving Counter Value

```
Counter errorCounter = job.getCounters().findCounter(COUNTERS
    .ERROR_COUNT);
```

1.9.2 User-Defined Counters

- Are defined by means of Java `enum`. Each application can define an arbitrary number of enums.
- Are incremented in the Mappers and Reducers.
- The global/final value of each counter is available at the end of the job. It is stored/printed by the Driver (at the end of the execution of the job).

The name of the enum is the group name and each enum has a number of “fields.” The enum’s fields are the counter names. In mappers and/or reducers counters are incremented by using the `increment()` method:

Incrementing Counter in Mapper/Reducer

```
context.getCounter(countername).increment(value);
```

The `getCounters()` and `findCounter()` methods are used by the Driver to retrieve the final values of the counters.

1.10 Summarization Patterns

1.10.1 Numerical Summarizations

Goal: Group records by key fields and calculate numerical aggregates (e.g., average, max, min, standard deviation) per group to provide a top-level view of large datasets.

Known Uses:

- Word count
- Record count per group
- Min/Max/Count per group
- Average/Median/Standard deviation per group

Structure:

- **Mappers:** Emit (key, value) pairs where the key represents grouping fields and the value represents the fields used to compute aggregate statistics.
- **Reducers:** Receive numerical values for each group and compute final statistics.
- **Combiners:** Useful for commutative and associative statistics to improve performance.

1.10.2 Inverted Index Summarizations

Goal: Build an index from the input data to support faster searches or data enrichment by mapping terms to a list of identifiers.

Structure:

- **Mappers:** Emit (key, value) pairs where the key is a keyword and the value is a unique identifier.
- **Reducers:** Concatenate identifiers for each keyword.
- **Combiners:** Not typically useful in this pattern.

1.10.3 Counting with Counters

Goal: Compute count summarizations of datasets and provide a high-level view.

Structure:

- **Mappers:** Increment counters directly; typically a map-only job with no reducers or combiners.
- **Driver:** Stores and prints the final counter results.

1.11 Filtering Patterns

1.11.1 Filtering

Goal: Filter out input records that are not of interest/keep only the ones that are of interest. Focus the analysis of the records of interest.

Structure

- **Mappers:** Output one (**key**, **value**) pair for each record that satisfies the enforced filtering rule. Key is associated with the primary key of the record. Value is associated with the selected record.
 - The input of the mapper is a set of records where Key = primary key and Value = record.
- **Reducers:** The reducer is useless in this pattern. A map-only job is executed (number of reduce set to 0).

1.11.2 Top K

Goal: Select a small set of top K records according to a ranking function. Focus on the most important records of the input data set.

Known Uses

- Outlier analysis (based on a ranking function).
- Select interesting data (based on a ranking function).

Structure

- **Mappers:** Each mapper initializes an in-mapper (local) top k list. k is usually small (e.g., 10). The current (local) top k-records of each mapper (i.e., instance of the mapper class) can be stored in main memory. Initialization is performed in the setup method of the mapper. The map function updates the current in-mapper top k list. The cleanup method emits the k (**key**, **value**) pairs associated with the in-mapper local top k records. Key is the “null key” and Value is an in-mapper top k record.
- **Input Data:** We work under the assumption that each input record to be ranked in the top K list is unique. For example, a list of items without duplicates. If duplicates had been present, an aggregating map-reduce job could have been performed to generate unique items.
- **Reducer:** A single reducer must be instantiated (i.e., one single instance of the reducer class). One single global view over the intermediate results emitted by the mappers to compute the final top k records. It computes the final top k list by merging the local lists emitted by the mappers. All input (**key**, **value**) pairs have the same key, hence, the reduce method is called only once.

1.11.3 Distinct

Goal: Find a unique set of values/records. In some applications, duplicate records are useless.

Known Uses

- Duplicate data removal.
- Distinct value selection.

Structure

- **Mappers:** Emit one (key, value) pair for each input record. Key = input record and Value = null value.
- **Reducers:** Emit one (key, value) pair for each input (key, list of values) pair. Key = input key, i.e., input record and Value = null value.

1.11.4 Multiple Inputs

Goal: In some applications, data are read from two or more datasets. These datasets may have different formats. Hadoop allows reading data from multiple inputs (datasets) with different formats.

Key Points:

- One mapper is specified for each input dataset.
- The key-value pairs emitted by the mappers must have consistent data types.

In the Driver:

- Use the `addInputPath` method of the `MultipleInputs` class multiple times:
 - Add one input path at a time.
 - Specify the input format class for each path.
 - Specify the Mapper class associated with each path.

Example

```
MultipleInputs.addInputPath(job, new Path(args[1]),
    TextInputFormat.class, Mapper1.class);
MultipleInputs.addInputPath(job, new Path(args[2]),
    TextInputFormat.class, Mapper2.class);
```

- `args[1]` and `args[2]` are input paths.
- Data from both paths are read using the `TextInputFormat` class.
- `Mapper1` manages the first path; `Mapper2` manages the second.

1.11.5 Multiple Outputs

Goal: Store the output key-value pairs of a MapReduce job in different files. Each file contains a specific subset of key-value pairs based on predefined rules.

Key Points:

- Each output file has a prefix to indicate its content (e.g., `part-` by default).
- All files are stored in a single output directory.
- The `MultipleOutputs` class allows specifying the prefixes of output files.

1.11.6 Driver Implementation

Use the `addNamedOutput` method of `MultipleOutputs` in the Driver:

- Specify the prefix of each output file.
- The method has the following parameters:
 - The job object.
 - The “name/prefix” for `MultipleOutputs`.
 - The `OutputFormat` class.
 - The key and value output data type classes.

Example

```
MultipleOutputs.addNamedOutput(job, "hightemp",
    TextOutputFormat.class, Text.class, NullWritable.class);
MultipleOutputs.addNamedOutput(job, "normaltemp",
    TextOutputFormat.class, Text.class, NullWritable.class);
```

This example defines:

- Output files with the prefix `hightemp`.
- Output files with the prefix `normaltemp`.

1.11.7 Map-Only Example

For a map-only job:

1. Define a private `MultipleOutputs` variable in the mapper:

```
private MultipleOutputs<Text, NullWritable> mos = null;
```

2. Create an instance of the `MultipleOutputs` class in the `setup` method:

```
mos = new MultipleOutputs<Text, NullWritable>(context);
```

3. Use the `write` method of the `MultipleOutputs` object in the `map` method:

```
mos.write("hightemp", key, value);
mos.write("normaltemp", key, value);
```

4. Close the `MultipleOutputs` object in the `cleanup` method:

```
mos.close();
```

This example writes:

- Key-value pairs with the prefix `hightemp-`.

- Key-value pairs with the prefix `normaltemp-`.

1.12 Data Organization Patterns

1.12.1 Binning

Goal: Organize/move the input records into categories.

Intent: Partition a big data set into distinct, smaller data sets (“bins”) containing similar records. Each partition is usually the input of a following analysis.

Structure

- **Driver:** Sets the list of “bins/output files” by means of `MultipleOutputs`.
- **Mappers:** For each input (`key`, `value`) pair:
 - Select the output bin/file associated with it.
 - Emit a (`key`, `value`) pair in that file, where:
 - * `key` of the emitted pair = `key` of the input pair.
 - * `value` of the emitted pair = `value` of the input pair.
- No combiner or reducer is used in this pattern.

1.12.2 Shuffling

Goal: Randomize the order of the data (records).

Motivation: Randomize the order of the data for anonymization reasons or for selecting a subset of random data (records).

Structure

- **Mappers:** Emit one (`key`, `value`) for each input record, where:
 - `key` is a random key (i.e., a random number).
 - `value` is the input record.
- **Reducers:** Emit one (`key`, `value`) pair for each `value` in `[list-of-values]` of the input (`key`, `[list-of-values]`) pair.

1.13 Metapatterns

1.13.1 Job Chaining

Goal: Execute a sequence of jobs (synchronizing them).

Intent: Manage the workflow of complex applications based on many phases (iterations). Each phase is associated with a different MapReduce Job (i.e., one sub-application). The output of a phase is the input of the next one.

Structure

- The (single) Driver contains the workflow of the application and executes the jobs in the proper order.
- **Mappers, reducers, and combiners:** Each phase of the complex application is implemented by a MapReduce Job, i.e., it is associated with a mapper, a reducer (and a combiner, if it is useful).

1.14 Join Patterns

1.14.1 Reduce-Side Natural Join

Goal: Join the content of two relations (i.e., relational tables). Both tables are large.

Motivation: The join operation is useful in many applications.

Structure

- There are two mapper classes, one for each table.
- **Mappers:** Emit one (key, value) pair for each input record, where:
 - **key** is the value of the common attribute(s).
 - **value** is the concatenation of the name of the table of the current record and the content of the current record.
- **Reducers:**
 - Iterate over the values associated with each **key** (value of the common attributes).
 - Compute the “local natural join” for the current **key**.
 - Generate a copy for each pair of values such that one record is from the first table and the other is from the second table.

Example: Suppose you want to join the following tables:

- **Users** with schema `userid, name, surname`.
- **Likes** with schema `userid, movieGenre`.

For instance, the record:

- `userid=u1, name=Paolo, surname=Garza` in **Users** generates:
`(userid=u1, "Users:name=Paolo,surname=Garza")`
- `userid=u1, movieGenre=horror` in **Likes** generates:
`(userid=u1, "Likes:movieGenre=horror")`

The reducer processes:

```
(userid=u1, ["Users:name=Paolo,surname=Garza",
             "Likes:movieGenre=horror",
             "Likes:movieGenre=adventure"])
```

It generates:

- (userid=u1, "name=Paolo,surname=Garza, genre=horror")
- (userid=u1, "name=Paolo,surname=Garza, genre=adventure")

1.14.2 Map-Side Natural Join

Goal: Join the content of two relations (i.e., relational tables). One table is large. The other is small enough to be completely loaded in main memory.

Motivation: The join operation is useful in many applications, and frequently one of the two tables is small.

Structure

- **Mapper class:**
 - Processes the content of the large table.
 - Receives one input (**key**, **value**) pair for each record of the large table.
 - Joins it with the “small” table.
- The distributed cache approach is used to provide a copy of the small table to all mappers.
- Each mapper performs the “local natural join” between the current record (of the large table) it is processing and the records of the small table (that is in the distributed cache).
- The content of the small table (file) is loaded in the main memory of each mapper during the execution of its **setup** method.

2 Theory Spark

2.1 RDD Creation

2.1.1 Create RDDs from Files

An RDD can be created from an input textual file using the `textFile(name)` method of the `SparkContext` class. The returned RDD is an RDD of Strings, where each line of the input file corresponds to an object (a string) in the RDD. By default, if the input file is stored in HDFS, the number of partitions of the created RDD will match the number of HDFS blocks used to store the file. Developers can manually set the minimum number of partitions using the `textFile(name, minPartitions)` method.

Create RDD from File

```
# Build an RDD of strings from the input textual file myfile.
txt
# Each element of the RDD is a line of the input file
inputFile = "myfile.txt"
lines = sc.textFile(inputFile)
lines = sc.textFile(inputFile, 4)
```

2.1.2 Create RDDs from a Local Python Collection

An RDD can also be created from a local Python collection (e.g., a list of local Python objects) using the `parallelize(c)` method of the `SparkContext` class. The created RDD will have one element for each object in the input collection. Spark automatically determines the number of partitions based on the cluster's characteristics. However, developers can manually set the number of partitions using `parallelize(c, numSlices)`.

Create RDD from Local Python Collection

```
# Create a local python list
inputList = ['First_element', 'Second_element', 'Third_
element']
# Build an RDD of Strings from the local list.
# The number of partitions is set automatically by Spark
# There is one element of the RDD for each element of the
local list
distRDDList = sc.parallelize(inputList)
distRDDList = sc.parallelize(inputList, 3)
```

2.1.3 Save RDDs

An RDD can be stored in a textual (HDFS) file using the `saveAsTextFile(path)` method of the RDD class. The method is invoked on the RDD that we want to store in the output folder. Each object of the RDD is stored in one line of the output files in the specified folder.

Save RDD to Text File

```
# Store the content of linesRDD in the output folder
# Each element of the RDD is stored in one line
# of the textual files of the output folder
outputPath="risFolder/"
linesRDD.saveAsTextFile(outputPath)
```

2.1.4 Retrieve the Content of RDDs

The content of an RDD can be retrieved from the nodes of the cluster and stored in a local Python variable on the Driver using the `collect()` method. This method returns a local Python list containing the elements of the RDD.

Warning: If the RDD is large, collecting it to the Driver can lead to memory issues.

Retrieve RDD Content

```
# Retrieve the content of the linesRDD and store it in a
# local python list
# The local python list contains a copy of each element of
# linesRDD
contentOfLines = linesRDD.collect()
```

2.2 Transformations and Actions

Transformations are operations on RDDs that return a new RDD. **Actions** are operations that return results to the Driver program, such as returning local Python variables or writing the result to storage (output files or folders).

2.2.1 Example of a Solution Based on def

This example shows how to define a function to filter elements of interest from an RDD, such as selecting rows containing the word "error" from a log file.

Filter RDD Based on Custom Function

```
# Define the content of the Boolean function that is applied
# to select the elements of interest
def myFunction(l):
    if l.find('error') >= 0: return True
    else: return False
# Read the content of a log file
inputRDD = sc.textFile("log.txt")
# Select the rows containing the word "error"
errorsRDD = inputRDD.filter(myFunction)
```

2.3 Basic transformations based on one single RDD: Summary

All the examples reported in the following table are applied on an RDD of integers containing the following elements (i.e., values) [1, 2, 3, 3].

Transformation	Purpose	Example of applied function	Result
<code>filter(f)</code>	Return an RDD consisting only of the elements of the “input” RDD that pass the condition passed to filter. The “input” RDD and the new RDD have the same data type.	<code>filter(lambda x: x != 1)</code>	[2,3,3]
<code>map(f)</code>	Apply a function to each element in the RDD and return an RDD of the result. The applied function return one element for each element of the “input” RDD. The “input” RDD and the new RDD can have a different data type.	<code>map(lambda x: x+1)</code>	[2,3,4,4]
<code>flatMap(f)</code>	Apply a function to each element in the RDD and return an RDD of the result. The applied function return a set of elements (from 0 to many) for each element of the “input” RDD. The “input” RDD and the new RDD can have a different data type.	<code>flatMap(lambda x: list(range(x,4)))</code>	[1,2,3,2,3,3,3]
<code>distinct()</code>	Remove duplicates	<code>distinct()</code>	[1,2,3]
<code>sortBy(keyfunc)</code>	Return a new RDD containing the same values of the input RDD sorted in ascending order.	<code>sortBy(lambda v: v)</code>	[1,2,3,3]
<code>sample(withReplacement, fraction)</code>	Sample the content of the “input” RDD, with or without replacement, and return the selected sample. The “input” RDD and the new RDD have the same data type.	<code>sample(True, 0.2)</code>	Non deterministic

2.4 Basic transformations based on two RDDs: Summary

All the examples reported in the following tables are applied on the following two RDDs of integers: **inputRDD1** [1, 2, 2, 3, 3]; **inputRDD2** [3, 4, 5]

Transformation	Purpose	Example	Result
<code>union(other)</code>	Return a new RDD containing the union of the elements of the “input” RDD and the elements of the one passed as parameter to <code>union()</code> . Duplicate values are not removed. All the RDDs have the same data type.	<code>inputRDD1.union(inputRDD2)</code>	[1, 2, 2, 3, 3, 3, 4, 5]
<code>intersection(other)</code>	Return a new RDD containing the intersection of the elements of the “input” RDD and the elements of the one passed as parameter to <code>intersection()</code> . All the RDDs have the same data type.	<code>inputRDD1.intersection(inputRDD2)</code>	[3]
<code>subtract(other)</code>	Return a new RDD containing the elements appearing only in the “input” RDD and not in the one passed as parameter to <code>subtract()</code> . All the RDDs have the same data type.	<code>inputRDD1.subtract(inputRDD2)</code>	[1, 2, 2]
<code>cartesian(other)</code>	Return a new RDD containing the cartesian product of the elements of the “input” RDD and the elements of the one passed as parameter to <code>cartesian()</code> . All the RDDs have the same data type.	<code>inputRDD1.cartesian(inputRDD2)</code>	[(1, 3), (1,4), ..., (3,5)]

2.5 Basic actions: Summary

All the examples reported in the following tables are applied on `inputRDD` that is an RDD of integers containing the following elements (i.e., values) `[1, 2, 3, 3]`.

Action	Purpose	Example	Result
<code>collect()</code>	Return a Python list containing all the elements of the RDD on which it is applied. The objects of the RDD and objects of the returned list are objects of the same class.	<code>inputRDD.collect()</code>	<code>[1,2,3,3]</code>
<code>count()</code>	Return the number of elements of the RDD.	<code>inputRDD.count()</code>	4
<code>countByValue()</code>	Return a Map object containing the information about the number of times each element occurs in the RDD.	<code>inputRDD.countByValue()</code>	<code>[(1, 1), (2, 1), (3, 2)]</code>
<code>take(num)</code>	Return a Python list containing the first num elements of the RDD. The objects of the RDD and objects of the returned list are objects of the same class.	<code>inputRDD.take(2)</code>	<code>[1,2]</code>
<code>first()</code>	Return the first element of the RDD.	<code>first()</code>	1
<code>top(num)</code>	Return a Python list containing the top num elements of the RDD based on the default sort order/comparator of the objects. The objects of the RDD and objects of the returned list are objects of the same class.	<code>inputRDD.top(2)</code>	<code>[3,3]</code>
<code>takeSample(withReplacement, num)</code>	Return a Python list containing a random sample of size n of the RDD. The objects of the RDD and objects of the returned list are objects of the same class.	<code>inputRDD.takeSample(False, 1)</code>	Nondeterministic
<code>reduce(f)</code>	Return a single Python object obtained by combining the values of the objects of the RDD by using a user-provided “function”. The provided “function” must be associative and commutative. The object returned by the method and the objects of the RDD belong to the same class.	<code>inputRDD.reduce(lambda e1, e2: e1+e2)</code>	9
<code>fold(zeroValue, op)</code>	Same as reduce but with the provided zero value.	<code>inputRDD.fold(0, lambda v1, v2: v1+v2)</code>	9
<code>aggregate(zeroValue, seqOp, combOp)</code>	Similar to reduce() but used to return a different type.	<code>inputRDD.aggregate(zeroValue, lambda acc, e: (acc[0]+e, acc[1]+1), lambda p1, p2: (p1[0]+p2[0], p1[1]+p2[1]))</code>	(9,4)

2.6 Transformations on RDDs of key-value pairs: Summary

All the examples reported in the following tables are applied on an RDD of pairs containing the following tuples (pairs): $[("k1", 2), ("k3", 4), ("k3", 6)]$. The key of each tuple is a string. The value of each tuple is an integer

Transformations	Purpose	Example	Result
<code>reduceByKey(f)</code>	Return an RDD of pairs containing one pair for each key of the "input" RDD of pairs. The value of each pair of the new RDD of pairs is obtained by combining the values of the input RDD associated with the same key. The "input" RDD of pairs and the new RDD of pairs have the same data type.	<code>reduceByKey(lambda v1, v2: v1+v2)</code>	$[("k1", 2), ("k3", 10)]$
<code>foldByKey(zeroValue, op)</code>	Similar to the <code>reduceByKey()</code> transformation. However, <code>foldByKey()</code> is characterized also by a zero value.	<code>foldByKey(0, lambda v1, v2: v1+v2)</code>	$[("k1", 2), ("k3", 10)]$
<code>combineByKey(createCombiner, mergeValue, mergeCombiner)</code>	Return an RDD of key-value pairs containing one pair for each key of the "input" RDD of pairs. The value of each pair of the new RDD is obtained by combining the values of the input RDD associated with the same key. The values of the "input" RDD of pairs and the values of the new (returned) RDD of pairs can be characterized by different data types.	<code>combineByKey(lambda e: (e, 1), lambda c, e: (c[0]+e, c[1]+1), lambda c1, c2: (c1[0]+c2[0], c1[1]+c2[1]))</code>	$[("k1", (2, 1)), ("k3", (10, 2))]$
<code>groupByKey()</code>	Return an RDD of pairs containing one pair for each key of the "input" RDD of pairs. The value of each pair of the new RDD of pairs is a "collection" containing all the values of the input RDD associated with one of the input keys.	<code>groupByKey()</code>	$[("k1", [2]), ("k3", [4, 6])]$
<code>mapValues(f)</code>	Apply a function over each pair of an RDD of pairs and return a new RDD of pairs. The applied function returns one pair for each pair of the "input" RDD of pairs. The function is applied only on the value part without changing the key. The values of the "input" RDD and the values of new RDD can have different data types.	<code>mapValues(lambda v: v+1)</code>	$[("k1", 3), ("k3", 5), ("k3", 7)]$
<code>flatMapValues(f)</code>	Apply a function over each pair of an RDD of pairs and return a new RDD of pairs. The applied function returns a set of pairs (from 0 to many) for each pair of the "input" RDD of pairs. The function is applied only on the value part without changing the key. The values of the "input" RDD and the values of new RDD can have different data types.	<code>flatMapValues(lambda v: list(range(v,6)))</code>	$[("k1", 2), ("k1", 3), ("k1", 4), ("k1", 5), ("k3", 4), ("k3", 5)]$
<code>keys()</code>	Return an RDD containing the keys of the input pairRDD.	<code>keys()</code>	$["k1", "k3", "k3"]$
<code>values()</code>	Return an RDD containing the values of the input pairRDD.	<code>values()</code>	$[2, 4, 6]$
<code>sortByKey()</code>	Return a PairRDD sorted by key. The "input" PairRDD and the new PairRDD have the same data type.	<code>sortByKey()</code>	$[("k1", 2), ("k3", 4), ("k3", 6)]$

2.7 Transformations on two RDDs of key-value pairs

All the examples reported in the following tables are applied on the following two RDDs of key-value pairs: **inputRDD1**: [(*k1'*, 2), (*k3'*, 4), (*k3'*, 6)]; **inputRDD2**: [(*k3'*, 9)]

Transformation	Purpose	Example	Result
<code>subtractByKey(other)</code>	Return a new RDD of key-value pairs. The returned pairs are those of input RDD on which the method is invoked such that the key part does not occur in the keys of the RDD that is passed as parameter. The values are not considered to take the decision.	<code>inputRDD1.subtractByKey(inputRDD2)</code>	[(<i>k1'</i> , 2)]
<code>join(other)</code>	Return a new RDD of pairs corresponding to join of the two input RDDs. The join is based on the value of the key.	<code>inputRDD1.join(inputRDD2)</code>	[(<i>k3'</i> , (4,9)), (<i>k3'</i> , (6,9))]
<code>cogroup(other)</code>	For each key k in one of the two input RDDs of pairs, return a pair (k, tuple), where tuple contains: - the list (iterable) of values of the first input RDD associated with key k - the list (iterable) of values of the second input RDD associated with key k	<code>inputRDD1.cogroup(inputRDD2)</code>	[(<i>k1'</i> , ([2], [])), (<i>k3'</i> , ([4, 6], [9]))]

2.8 Actions on RDDs of key-value pairs: Summary

All the examples reported in the following tables are applied on the following RDD of key-value pairs: **inputRDD**: [(*k1'*, 2), (*k3'*, 4), (*k3'*, 6)]

Transformation	Purpose	Example	Result
<code>countByKey()</code>	Return a local python dictionary containing the number of elements in the input RDD for each key of the input RDD of pairs	<code>inputRDD.countByKey()</code>	{(<i>k1'</i> , 1), (<i>k3'</i> , 2)}
<code>collectAsMap()</code>	Return a local python dictionary containing the pairs of the input RDD of pairs	<code>inputRDD.collectAsMap()</code>	{(<i>k1'</i> , 2), (<i>k3'</i> , 6)} or {(<i>k1'</i> , 2), (<i>k3'</i> , 4)} depending on the order of the pairs in the input RDD of pairs
<code>lookup(key)</code>	Return a local python list containing all the values associated with the key specified as parameter	<code>inputRDD.lookup('k3')</code>	[4,6]

2.9 RDDs of numbers: Summary

All the examples reported in the following are applied on inputRDD that is an RDD containing the following double values [1.5, 3.5, 2.0]

Action	Purpose	Example	Result
<code>sum()</code>	Return the sum over the values of the inputRDD	<code>inputRDD.sum()</code>	7
<code>mean()</code>	Return the mean value	<code>inputRDD.mean()</code>	2.3333
<code>stdev()</code>	Return the standard deviation computed over the values of the inputRDD	<code>inputRDD.stdev()</code>	0.8498
<code>variance()</code>	Return the variance computed over the values of the inputRDD	<code>inputRDD.variance()</code>	0.7223
<code>max()</code>	Return the maximum value	<code>inputRDD.max()</code>	3.5
<code>min()</code>	Return the minimum value	<code>inputRDD.min()</code>	1.5

2.10 Persistence and Cache

Spark computes the content of an RDD each time an action is invoked on it. When you ask Spark to persist/cache an RDD, each node stores the content of its partitions in memory and

reuses them in other actions on that RDD/dataset (or RDDs derived from it). Spark supports several storage levels: the storage level is used to specify if the content of the RDD is stored in the main memory of the nodes, on the local disks of the nodes or partially in the main memory and partially on disk

Storage Level	Meaning
MEMORY_ONLY	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
MEMORY_AND_DISK	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on (local) disk, and read them from there when they're needed.
DISK_ONLY	Store the RDD partitions only on disk.
MEMORY_ONLY_2, MEMORY_AND_DISK_2, etc.	Same as the levels above, but replicate each partition on two cluster nodes.
OFF_HEAP (experimental)	Similar to MEMORY_ONLY, but store the data in off-heap memory. This requires off-heap memory to be enabled.

You can mark an RDD to be **persisted** by using the `persist(storageLevel)` method of the RDD class. You can cache an RDD by using the `cache()` method of the RDD class, which persists the RDD with the storage level `MEMORY_ONLY`. Note that both `persist` and `cache` return a new RDD. You can manually remove an RDD from the cache by using the `unpersist()` method of the RDD class.

Example: Create an RDD from a textual file containing a list of words. One word for each line. Print on the standard output:

- The number of lines in the input file
- The number of distinct words

Count words and distinct words

```
# Read the content of a textual file
# and cache the associated RDD
inputRDD = sc.textFile("words.txt").cache()
print("Number_of_words:", inputRDD.count())
print("Number_of_distinct_words:", inputRDD.distinct().count())
```

2.11 Accumulators

When a “function” passed to a Spark operation is executed on a remote cluster node, it works on separate copies of all the variables used in the function. Spark provides a type of shared variables called **accumulators**. Accumulators are shared variables that are only “added” to through an associative operation and can therefore be efficiently supported in parallel. Accumulators are defined and initialized by using the `accumulator(value)` method of the `SparkContext` class. The value of an accumulator can be “increased” by using the `add(value)` method of the `Accumulator` class. The final value of an accumulator can be retrieved in the driver program by using the `value` of the `Accumulator` class.

Example: Create an RDD from a textual file containing a list of email addresses (one email for each line). Select the lines containing a valid email and store them in an HDFS file. In this example, an email is considered valid if it contains the @ symbol. Print also, on the standard output, the number of invalid emails.

Count valid and invalid emails

```
# Define an accumulator. Initialize it to 0
invalidEmails = sc.accumulator(0)
# Read the content of the input textual file
emailsRDD = sc.textFile("emails.txt")
# Define the filtering function
def validEmailFunc(line):
    if (line.find('@') < 0):
        invalidEmails.add(1)
        return False
    else:
        return True
# Select only valid emails
# Count also the number of invalid emails
validEmailsRDD = emailsRDD.filter(validEmailFunc)
```

2.12 Personalized Accumulators

Programmers can define accumulators based on new data types (different from integers and floats). To define a new accumulator data type of type T, the programmer must define a class subclassing the `AccumulatorParam` interface.

2.13 Broadcast Variables

A **broadcast** variable is a read-only (small/medium) shared variable that is instantiated in the driver and sent to all worker nodes that use it in one or more Spark operations. A broadcast variable is sent only one time to each executor that uses that variable in at least one Spark operation. Broadcast variables are objects of type **Broadcast**. A broadcast variable (of type T) is defined in the driver by using the `broadcast(value)` method of the `SparkContext` class. The value of a broadcast variable (of type T) is retrieved (usually in transformations) by using `value` of the `Broadcast` class.

Example: Create an RDD from a textual file containing a dictionary of pairs (word, integer value). One pair for each line. Suppose the content of this first file is large but can be stored in main-memory. Create an RDD from a textual file containing a set of words. A sentence (set of words) for each line. Transform the content of the second file mapping each word to an integer based on the dictionary contained in the first file. Store the result in an HDFS file.

First file (dictionary):

- java 1
- spark 2

- test 3

Second file (the text to transform):

- java spark
- spark test java

Output file:

- 1 2
- 2 3 1

Broadcast dictionary and map words

```
# Read the content of the dictionary from the first file and
# map each line to a pair (word, integer value)
dictionaryRDD = sc.textFile("dictionary.txt").map(lambda line
:
(line.split("_")[0], line.split("_")[1]))
# Create a broadcast variable based on the content of
  dictionaryRDD.
# Pay attention that a broadcast variable can be instantiated
  only
# by passing as parameter a local variable and not an RDD.
# Hence, the collectAsMap method is used to retrieve the
  content of the
# RDD and store it in the dictionary variable
dictionary = dictionaryRDD.collectAsMap()
# Broadcast dictionary
dictionaryBroadcast = sc.broadcast(dictionary)
```

3 Additional Information for MapReduce

These are some additional insights I found particularly useful after reviewing the labs and exercises on the course's website.

3.1 Using Placeholders in the Mapper Phase for Counting Occurrences

In the mapper phase of a MapReduce job, if you don't have a specific `OutputValue` to extract from the file, you can use a placeholder like `new IntWritable(1)` to indicate a single occurrence. This strategy is commonly used in the WordCount problem, where each word is counted by emitting key-value pairs such as `(word, 1)`. You can apply the same approach in the filtering pattern, such as when analyzing PM10 pollution data. If you want to count the number of days on which a sensor's reading was above a certain threshold, you can output a pair in the mapper phase like this: `context.write(new Text(sensor_id), new IntWritable(1));`. This means that for each day the sensor exceeds the threshold, the mapper will emit the sensor's ID as the key and 1 as the value, indicating one occurrence. The reducer will then sum these values to determine the total number of days that the sensor exceeded the threshold. This pattern is efficient for cases where the goal is to count occurrences rather than extract or process additional data from the input.

3.2 Using Custom Data Types to Preserve Key-Value Relationships in Map and Reduce Phases

In the **map phase**, if you need to preserve the relationship between a chosen key and multiple associated values, you can use a **custom data type**. This approach allows you to encapsulate the key-value relationship within a structured format. Later, during the **reduce phase**, you can leverage appropriately implemented methods in the custom data type to retrieve the necessary values without losing their connection to the original key. This technique ensures that complex relationships between keys and values are maintained throughout the processing pipeline. (*See Lab 4 for more details.*)

3.3 Types for Mapper, Driver and Reducer

Primitive Types

- **IntWritable**: Represents an integer (similar to `int` in Java).
- **LongWritable**: Represents a long integer (`long` in Java).
- **FloatWritable**: Represents a floating-point number (`float` in Java).
- **DoubleWritable**: Represents a double-precision floating-point number (`double`

in Java).

- **BooleanWritable**: Represents a boolean value (`boolean` in Java).

Custom Writable

You can create custom types by implementing the `Writable` interface and defining the `write` and `readFields` methods.

Text

Text: A Writable implementation for strings (similar to `String` in Java). Frequently used for textual keys or values.

NullWritable

NullWritable: A placeholder type that signifies the absence of a key or value (useful when the output doesn't require a key or value).

3.4 Guide: When to Use setup and cleanup with Custom Data Types and Data Structures in Hadoop

What Are setup and cleanup?

setup: A method called **once** at the start of a Mapper/Reducer task to initialize resources or data structures.

cleanup: A method called **once** at the end of a Mapper/Reducer task to finalize processing or release resources.

When NOT to Use setup and cleanup

1. **Stateless Processing:** If your logic does not require maintaining state across input records (e.g., simple transformations or one-off operations in `map` or `reduce`).
2. **Local Data Use:** If your data type is small, simple, and only used within a single call to `map` or `reduce`.

When to Use setup

Use `setup` when you need to:

1. **Initialize Shared Data Structures:** If your custom data type or data structure (e.g., `HashMap`, `ArrayList`) is needed across multiple calls to the `map` or `reduce` method.
2. **Load Reference Data:** When you need to load external configurations, reference files, or constants that will be reused throughout the task.
3. **Avoid Reinitializing Resources:** Prevent repeated initialization of objects that are expensive to create (e.g., database connections).

When to Use cleanup

Use `cleanup` when you need to:

1. **Finalize and Emit Aggregated Results:** Process or write the contents of a data structure (e.g., `HashMap`, `ArrayList`) after all input records have been processed.
2. **Batch Results to Reduce I/O:** Write aggregated or buffered results once at the end instead of emitting data for every input record during the `map` or `reduce` method.
3. **Release Resources:** Close files, connections, or clean up memory-intensive objects.

Example of using of setup and cleanup with HashMap

```

class MapperBigData extends
    Mapper<LongWritable, // Input key type
        Text, // Input value type
        Text, // Output key type
        SumCount> { // Output value type

    HashMap<String, SumCount> statistics;

    protected void setup(Context context) {
        statistics = new HashMap<String, SumCount>();
    }

    protected void map(LongWritable key, // Input key type
        Text value, // Input value type
        Context context) throws IOException,
        InterruptedException {

        SumCount currentStat;
        String record = value.toString();
        // Split each record by using the field separator
        // fields[0]= first attribute - sensor id
        // fields[1]= second attribute - timestamp
        // fields[2]= third attribute - reading
        String[] fields = record.split(",");
        String sensorId = fields[0];
        float measure = Float.parseFloat(fields[2]);
        currentStat = statistics.get(sensorId);
        if (currentStat == null) {
            currentStat = new SumCount();
            currentStat.setCount(1);
            currentStat.setSum(measure);
            statistics.put(new String(sensorId), currentStat);
        } else {
            currentStat.setCount(currentStat.getCount() + 1);
            currentStat.setSum(currentStat.getSum() + measure);
            statistics.put(new String(sensorId), currentStat);
        }
    }

    protected void cleanup(Context context) throws IOException,
        InterruptedException {
        for (Entry<String, SumCount> pair : statistics.entrySet()) {
            context.write(new Text(pair.getKey()), pair.getValue());
        }
    }
}

```

Reducer

```

class ReducerBigData extends Reducer<
    Text,                // Input key type
    SumCount,           // Input value type
    Text,                // Output key type
    SumCount> {          // Output value type

    @Override
    protected void reduce(
        Text key, // Input key type
        Iterable<SumCount> values, // Input value type
        Context context) throws IOException,
        InterruptedException {

        int count=0;
        float sum=0;
        // Iterate over the set of values and sum them.
        // Count also the number of values
        for (SumCount value : values) {
            sum=sum+value.getSum();
            count=count+value.getCount();
        }
        SumCount sumCountPerSensor= new SumCount();
        sumCountPerSensor.setCount(count);
        sumCountPerSensor.setSum(sum);
        // Emits pair (sensor_id, sum-count = average)
        context.write(new Text(key), sumCountPerSensor);
    }
}

```

Example of using of setup and cleanup for a Top 1

```

protected void setup(Context context) {
    // for each mapper, top1 is used to store the information
    // about the top1 date-income of the subset of lines
    // analyzed by the mapper
    top1 = new DateIncome();
    top1.setIncome(Float.MIN_VALUE);
    top1.setDate(null);
}

protected void map(Text key, // Input key type
                  Text value, // Input value type
                  Context context) throws IOException,
                  InterruptedException {

    String date = new String(key.toString());
    float dailyIncome = Float.parseFloat(value.toString());
    // Check if the current income is the top 1 income among
    // the ones analyzed by this mapper. In case of tie (same
    // income value) the earliest date is considered.
    if ( dailyIncome > top1.getIncome() || (dailyIncome ==
        top1.getIncome() && date.compareTo(top1.getDate()) <
        0)) {
        // The current line is the current top 1 income value
        // . Store date and income in top1
        top1 = new DateIncome();
        top1.setDate(date);
        top1.setIncome(dailyIncome);
    }
}

protected void cleanup(Context context) throws IOException,
    InterruptedException {
    // Emit the top1 date and income related to this mapper
    context.write(NullWritable.get(), top1);
}

```

Reducer

```
// The reduce method is called only once in this approach.
// All the key-value pairs emitted by the mappers have the
// same key (NullWritable.get())

@Override
protected void reduce(
    NullWritable key, // Input key type
    Iterable<DateIncome> values, // Input value
    type
    Context context) throws IOException,
    InterruptedException {

    String date;
    float dailyIncome;
    DateIncome globalTop1 = new DateIncome();
    globalTop1.setIncome(Float.MIN_VALUE);
    globalTop1.setDate(null);
    // Iterate over the set of values and select the top
    // 1 income and the related date
    for (DateIncome value : values) {
        date = value.getDate();
        dailyIncome = value.getIncome();
        if (dailyIncome > globalTop1.getIncome() || (
            dailyIncome == globalTop1.getIncome() && date.
            compareTo(globalTop1.getDate()) < 0)) {
            // The current line is the current top 1
            // income value. Store date and income in
            // globalTop1
            globalTop1 = new DateIncome();
            globalTop1.setDate(date);
            globalTop1.setIncome(dailyIncome);
        }
    }
    // Emit pair (date, income) associated with top 1 income
    context.write(new Text(globalTop1.getDate()), new
        FloatWritable(globalTop1.getIncome()));
}
}
```

Example of using the setup method in the reducer with a shared value

```

//Input: a collection of news (textual files)
//Output: List of distinct words occurring in the collection
         associated with a set of unique integers. Each word is
         associated with a unique integer (and viceversa)

int wordId;

protected void setup(Context context) {
    // Initialize the variable that is used to remember how
    // many words have been already mapped to an integer (i.e
    // ., it stores also the last integer value mapped with a
    // word)
    wordId = 0;
}

@Override
protected void reduce(Text key, // Input key type
    Iterable<NullWritable> values, // Input value type
    Context context) throws IOException,
    InterruptedException {
    // Emit the current word associated with the next
    // available integer
    wordId = wordId + 1;
    context.write(key, new IntWritable(wordId));
}

```

3.5 Useful pattern and function - Mapper

3.5.1 Splitting the sentence in a TextInputFile

Split each sentence into words. Use whitespace(s) as delimiter (a space, a tab, a line break, or a form feed). The split method returns an array of strings. Useful when the input is a `TextInputFile` and I want to extract each word from the text.

```
String[] words = value.toString().split("\\s+");
```

3.5.2 Splitting each record of a csv file by using the field separator

A similar approach can be applied when working with a CSV file where the fields are separated by commas (`,`).

```
// Split each record by using the field separator
// fields[0]= first attribute - sensor id
// fields[1]= second attribute - date
// fields[2]= third attribute - PM10 value
String[] fields = value.toString().split(",");
String sensorId = fields[0];
float PM10value = Float.parseFloat(fields[2]);
```

3.5.3 Extracting the value in a KeyValueTextInputFormat

In this example, the input is in the format of a `KeyValueTextInputFormat`, where the key represents the `zone,date` and the value represents the `threshold`. To correctly extract the `zone`, the `date` and the `threshold` from this input, the following approach should be used:

Example: How to extract the value in the correct way

```
// Extract zone and date from the key
String[] fields = key.toString().split(",");

String zone = fields[0];
String date = fields[1];
Double PM10Level = new Double(value.toString());
```

3.5.4 Emitting pair in a TextInputFile

To send the pairs to the reducer, iterate through the set of words using the loop:

```
// Iterate over the set of words
for(String word : words) {
    // Transform word case
    String cleanedWord = word.toLowerCase();
    // emit the pair (word, 1)
    context.write(new Text(cleanedWord), new IntWritable(1));
    // emit the pair (word, null)
    context.write(new Text(cleanedWord), NullWritable.get());
}
```

3.6 Useful pattern and function - Reducer

3.6.1 Operation on the set of values

Remember: the reduce function takes an input value of type `Iterable<T>`. Typically, you will work on this set of values to perform the desired operations.

Example of operation on the set of values

```
// Iterate over the set of values and sum them. Count also
// the number of values
//In this case I had an Iterable<FloatWritable> as input
// value type
for (FloatWritable value : values) {
    sum = sum + value.get();
    count = count + 1;
}
```

4 Additional Information for Spark

These are some additional insights I found particularly useful after reviewing the labs and exercises on the course's website.

4.1 Useful pattern and function

4.1.1 Extracting a Specific Field from an RDD

Use the following pattern to extract a specific field `n` (where `n` is the field number) from an RDD (e.g., `readingsRDD`) and store the values in another RDD (e.g., `valuesRDD`). In this example, the value is assumed to be a `float`, but you can adapt it to match your requirements.

```
valuesRDD = readingsRDD.map(lambda VReading: float(VReading.
    split(',') [n]))
```

When to use: Use this pattern when you need to extract a particular field from each record in an RDD and convert it into a new RDD. This is common when you need to isolate specific fields (such as numeric values) for further analysis.

4.1.2 `takeOrdered()`

Action that retrieves a specified number of elements from an RDD or DataFrame, ordering them according to a specified **natural** or **custom comparator**.

`RDD.takeOrdered(n, key=None)`

- **n:** The number of elements to retrieve.
- **key** (*optional*): A function used to specify the ordering. For example, `lambda x: -x` can be used to reverse the natural order. If you want to retrieve the **top k** values, you can use the **top(k)** action as `takeOrdered(k, lambda x: -x)`.

When to use: Use this pattern when you need to retrieve a specified number of the smallest (or largest, by reversing the key) elements from an RDD or DataFrame in an ordered manner. This is useful when you need to find, for example, the smallest or largest values in a dataset.

```
data = sc.parallelize([10, 5, 8, 3, 15, 7])
smallest = data.takeOrdered(3)
print(smallest) # Output: [3, 5, 7]
```

4.1.3 `list()`

Converts an iterable into a list, or creates an empty list if no arguments are provided. This method is useful in Spark to ensure that each iterable (e.g., tuples or generators) in the

RDD's values is converted into a mutable, indexable format. It is used in combination with `mapValues()`.

When to use: Use this pattern when you need to convert the values of an RDD into a list format, typically when working with complex data types or iterables inside the RDD that need to be transformed into a mutable list. It is commonly used with `mapValues()` to handle complex transformations.

```
questionsAnswersReformatted = questionsAnswersPairRDD.
    mapValues(lambda value: (list(value[0]), list(value[1])))
```

4.1.4 Retrieving the Maximum Value from ValuesRDD

To extract the maximum value from `ValuesRDD` and assign it to the variable `maxValue`, use one of the following patterns:

```
maxValue = valuesRDD.reduce(lambda value1, value2: max(value1
    , value2))
# Equivalent
maxValue = valuesRDD.takeOrdered(1, lambda n: -1*n)[0]
# Equivalent
maxValue = valuesRDD.top(1)[0]
```

When to use: Use this pattern when you want to retrieve the maximum value from an RDD (in this case, `ValuesRDD`), where each element represents a numeric value. It is useful after filtering the RDD to contain only relevant values. This operation will return the highest value in the RDD.

Important: Ensure that `ValuesRDD` contains only the relevant values. You may need to use the filtering pattern before applying this one, depending on the structure of your data.

4.1.5 Retrieving Lines Associated with the Maximum Value

If you have already applied the previous pattern to find the maximum value, and now you want to extract the line(s) corresponding to `maxValue`, use the following pattern. Here, `n` represents the field number.

```
selectedRecordsRDD = readingsRDD.filter(lambda Reading: float
    (Reading.split(',')[n]) == maxValue)
```

When to use: Use this pattern when you want to retrieve the entire record(s) that corresponds to the maximum value identified in the previous step. It is useful for extracting detailed data related to the maximum value.

4.1.6 Associating a Number to Each Value in an RDD

Use the following pattern to associate a number (e.g., 1) with each value extracted from the RDD. This approach can be applied to any RDD for pairing values with a specific number.

```
valuesRDD = readingsRDD.map(lambda Reading: (float(Reading.
    split(',')[n]), 1))
```

When to use: Use this pattern when you need to pair each value from an RDD with a specific number (e.g., a count). This is useful for counting occurrences of values or for further aggregation, such as calculating the frequency of each value.

4.1.7 Computing the Sum and the Avg

Important: Ensure that ValuesRDD contains only the relevant values. You may need to use the filtering pattern before applying this one, depending on the structure of your data.

```
# Compute the sum
sumValues = valuesRDD.reduce(lambda value1, value2: value1+
    value2)
# Count the number of lines of the input file
numLines = valuesRDD.count()
# Compute average
print("Average=", sumValues / numLines)
```

4.1.8 Creating an RDD with Paired Fields from a CSV File

Use this pattern to create an RDD containing pairs of fields extracted from a CSV file. Here, **n1** and **n2** represent the fields to be selected. This pattern is particularly useful when combined with the one described below.

```
valuesRDD = readingsRDD.map(lambda Reading: (Reading.split(',
    ')[n1], float(Reading.split(',')[n2])))
```

When to use: Use this pattern when you need to extract specific fields from a CSV file and store them as key-value pairs in an RDD. This is useful when you need to organize data by specific keys for further processing.

4.1.9 Computing the Maximum Value for Each Key

Use this pattern to compute the maximum value associated with each key in an RDD. This approach is ideal for grouping data by keys and identifying the highest value within each group.

```
MaxValuesRDD = valuesRDD.reduceByKey(lambda value1, value2:  
    max(value1, value2))
```

When to use: Use this pattern when you need to compute the maximum value for each key in an RDD. This is often used for grouping data by key and finding the highest value in each group, such as finding the highest score for each student.

5 Closed Question

These are some tips to solve the closed question exercises based on the passed exam

5.1 Similar behaviour in Spark Streaming

1. Identify the Operations in Each Stream:

For each DStream (e.g., `resADStream`, `resBDStream`, `resCDStream`), break down the sequence of operations applied. Look at the order of transformations such as `map()`, `filter()`, `reduce()`, and `window()`.

2. Determine How Windowing and Reduction Interact

- **Windowing before Reduce:** The window splits the data into chunks, and the reduction is applied within each chunk. This means each window operates independently.
- **Reduce before Windowing:** The entire stream is reduced to a single value first, and then windowing is applied. This can lead to fewer windows or potentially different outcomes, as the data is first aggregated before the windowing.

3. Compare the Transformation Sequences

After breaking down the transformations, check which DStreams have similar sequences of operations. Pay attention to where the `reduce()` and `window()` operations are applied. This helps in determining which DStreams will have the same result.

4. Multiple reduce operations

When you have multiple reduce operations in a Spark Streaming application, the one that occurs last (after all transformations like `windowing`, `filtering`, etc.) is the one that determines the final behavior of the DStream. This last reduction is the one that actually affects the result that gets printed or stored because it operates on the data that has been transformed up to that point.

Practical Tips

When dealing with operations involving both `filter()` and `reduce()`, it's important to consider the interaction between the two functions, as the order of execution can impact the result.

- If `reduce(max)` is combined with `filter(< value)`, the order of operations matters because `max` and the "less than" filter work in opposite directions. Filtering first reduces the available values, potentially changing the final maximum.
- If `reduce(max)` is combined with `filter(> value)`, the order does not affect the result, as both operations align in the same direction—selecting larger values.

Similarly:

- If `reduce(min)` is combined with `filter(> value)`, the order matters because `min` and "greater than" filtering work against each other. Filtering first removes smaller values, altering the final minimum.
- If `reduce(min)` is combined with `filter(< value)`, the order does not impact the result, as both operations focus on selecting smaller values.

To explain the operations and their effects, let's consider a simple example with the following sample input:

$$(1, 11), (9, 3)$$

- **window():** The `window()` operation aggregates elements based on a defined window size. In this case, the window operation combines the pairs $(1, 11)$ and $(9, 3)$ into one aggregated result:

$$(1, 11), (9, 3) \rightarrow (1, 11, 9, 3)$$

- **reduce(max or min):** The `reduce()` operation processes the aggregated window of elements, and depending on whether we use `max` or `min`, it computes the corresponding result for each item in the window:

– For `max`:

$$(1, 11), (9, 3) \rightarrow (11), (9)$$

– For `min`:

$$(1, 11), (9, 3) \rightarrow (1), (3)$$

- **filter(cond):** The `filter()` operation removes elements that don't meet a specified condition `cond`. If, for instance, we filter values greater than 10, we would get:

$$(1, 11), (9, 3) \rightarrow (11), (-)$$

5.2 General Strategy for Solving Spark Streaming Window-Based Exercises

1. Identify batch size and window parameters

```
ssc = StreamingContext(sc, 10) # Batch interval: 10s
resDStream = sumDStream.window(20, 10) # Window: 20s, Slide:
10s
```

2. Simulate data arrival within batch intervals

```
Time: 1s   -> "2"
Time: 3s   -> "2"
Time: 5s   -> "1"
Time: 12s  -> "4"
Time: 14s  -> "2"
```

Grouped into batches

Batch 1 (0s-10s) -> [2, 2, 1] -> Sum = 5
 Batch 2 (10s-20s) -> [4, 2] -> Sum = 6

3. Compute operations per batch

```
sumDStream = inputDStream.reduce(lambda v1, v2: v1 + v2)
```

4. Analyze the windowed computations

```
resDStream = sumDStream.window(20, 10)
```

Windows

Window at 10s -> 5
 Window at 20s -> 5 + 6 = 11

5. Determine output at the specified time

```
resDStream.pprint() # Prints 11 at 20s
```

5.3 Practical Steps to Solve Spark RDD Read Count Problems

These functions force Spark to recompute an RDD or read from disk if the RDD is not cached.

Basic Actions (Trigger Execution)

- `count()`: Computes the number of elements, triggering a full scan of the dataset.
- `collect()`: Brings all data to the driver, triggering execution.
- `take(n)`: Retrieves `n` elements, triggering execution.
- `reduce(func)`: Aggregates data using `func`, forcing computation. **Be careful:** `reduce` triggers a new data read, whereas `reduceByKey` does not.
- `first()`: Returns the first element, triggering a read.

Actions That Involve Writing Data

- `saveAsTextFile(path)`: Writes RDD to HDFS/disk, triggering a full read.
- `saveAsSequenceFile(path)`: Similar to `saveAsTextFile`, but for sequence files.
- `saveAsObjectFile(path)`: Stores RDD as serialized objects, forcing computation.

Actions That Apply Functions Per Partition

- `foreach(func)`: Applies `func` to each element, triggering execution.
- `foreachPartition(func)`: Applies `func` to each partition, forcing a read.

Union and Intersection transformation

`union()` itself doesn't trigger a read. Actions on the resulting `UnionRDD` will cause each RDD in the union to be read once, and thus, you should count one read for each RDD involved in the union when an action is performed on the union. So, if you perform an action on `UnionRDD`, and `Temp1RDD` and `Temp2RDD` are the RDDs involved, you should count two reads, one for each RDD. The same thing is valid for the `intersection()` transformation.

Handling the `cache()` Function

When the `cache()` function is applied to an RDD, all subsequent actions performed on that RDD will reuse the cached data instead of triggering a new read operation. However, any actions executed before the `cache()` call will still cause the RDD to be read from the source.

5.4 Determine the number of mapper instantiated

Example Walkthrough:

Given:

- `Temperatures2019.txt` = 1030MB
- `Temperatures2020.txt` = 1018MB
- HDFS Block Size = 1024MB

Step 1: Calculate Number of Blocks

Temperatures2019.txt:

- Size: 1030MB
- Number of blocks = $\lceil \frac{1030}{1024} \rceil = 2$ blocks (1st block = 1024MB, 2nd block = 6MB)

Temperatures2020.txt:

- Size: 1018MB
- Number of blocks = $\lceil \frac{1018}{1024} \rceil = 2$ blocks (1st block = 1024MB, 2nd block = 94MB)

Step 2: Total Number of Blocks

- Total blocks = 2 (`Temperatures2019.txt`) + 2 (`Temperatures2020.txt`) = 4 blocks.

Step 3: Number of Mappers

- Total number of mappers = 4 (since each block is processed by a mapper).

5.5 Computation of reduce method invocations

The `reduce` method is invoked once for each unique key emitted by the `map` function. Therefore, the number of times the `reduce` method is called corresponds to the number of distinct keys.

Example:

```
(Luca,4)
(Luca,4)
(Luca,4)
(Carmen,6)
(Danilo,6)
(Carmen,6)
(Pablo,5)
(Pablo,5)
```

After the shuffle and sort phase, the key-value pairs are grouped by key:

```
(Luca, [4, 4, 4])
(Carmen, [6, 6])
(Danilo, [6])
(Pablo, [5, 5])
```

In this case, there are four distinct keys: Luca, Carmen, Danilo, and Pablo. Thus, the `reduce` method is invoked 4 times.

5.6 Computation of map method invocations

In a Hadoop MapReduce job, the `map` method is invoked once for each line in the input files. This means that to determine the total number of invocations, we simply need to count the number of lines across all input files.

Example

- **ProfilesItaly.txt**

```
Kevin,Rome
Luca,Rome
Candy,Naples
```

(3 lines)

- **ProfilesFrance.txt**

```
Paolo,Paris
Kevin,Nice
```

(2 lines)

Since each line is processed separately by the mapper, the total number of `map` method invocations is:

$$3 + 2 = 5$$

Thus, in this case, the `map` method is invoked **5 times** in total.

6 Hadoop Exam

These are some tips to solve the exercises about Hadoop based on the passed exam

6.1 Deciding the Number of Jobs in a Hadoop Application

Determining how many jobs to use in a Hadoop application depends on the problem's complexity, the data volume, and how intermediate results need to be processed. Here's a guide to help you decide:

Analyzing the Problem

Key Questions to Ask:

- Can the problem be solved in a single pass? If the mapper and reducer can produce a complete and final result, one job might suffice.
- Are there steps requiring separate processing stages (e.g., aggregation followed by sorting)? If intermediate data requires further processing, multiple jobs may be necessary.
- Are the intermediate results too large to handle in memory? In this case, it's better to write intermediate results to HDFS and split the work into separate jobs.

Structure of Intermediate Data

If intermediate data is **large or complex**, splitting the process into multiple jobs is often better.

Sorting and Top-K Problems

For tasks requiring global sorting (e.g., identifying the top-100 most frequent items), **two**

jobs are usually needed:

- The first aggregates the data.
- The second sorts the results and selects the top-k elements.

Complexity of the Computation

When is a single job enough?

- The data transformation is simple (e.g., counting occurrences or summing values).
- There's no need to sort or filter intermediate results further.

When are multiple jobs needed?

- There are **sequential dependencies** between different stages (e.g., the first job aggregates data, and the second selects the top-k results).
- Each computation stage generates output that requires additional processing (e.g., graph computations where weights need iterative updates).

Conclusion

- Always start with a **single job** if you think the problem can be solved that way.
- Split into multiple jobs when:
 - Intermediate data is large or complex.

- The problem involves sequential processing steps.
- Global computations (like sorting) are required.

6.2 Some basic instruction for mapper and reducer

Mapper

```
# Extract the fields from input file
String[] fields = value.toString().split(",");
# Parsing (in this case Double)
double salary = Double.parseDouble(fields[2]);
# When you have a 2 job application you may need to parse the
  value that derive from the reducer of job 1
int count = Integer.parseInt(value.toString());
# When you have to compute the greatest value typically in
  the mapper of the second job you will emit a pair like
  this
context.write(NullWritable.get(), new IntWritable(count));
# When you don't want to lose the link between to fields you
  can concatenate them
context.write(new Text(sid+", "+seasonNumber), new IntWritable
  (1));
```

Reducer

```
# Basic iteration on the values. In general I have Iterable<
  InputValueType> in the reduce method.
int count = 0;
for (InputValueType value : values) {
    # Compute the sum of all the values
    count += v.get();
}
context.write(key, new IntWritable(count));
# To extract a field that I have concatenated in the mapper
String sid = key.toString().split(",")[0];
```

6.3 Attribute with the Largest Number of Items

Exam request

Countries with the highest number of amateur observatories. The application considers only the amateur observatories and, for each country, computes the number of amateur observatories. Finally, it finds the country with the highest number of amateur observatories and stores it in the output folder. If more countries are associated with the maximum number of amateur observatories, sort the countries in alphabetical order and select the first. Store the

top country and the associated number of amateur observatories in the output HDFS folder (output pair (Country, Number of amateur observatories in that country)).

The data file has the following structure:

ObservatoryID,Name,Lat,Lon,Country,Continent,Amateur.

Since there is no direct field that specifies the number of amateur observatories, we need to count the lines that match the condition (i.e., when the observatory is an amateur one).

In Mapper 1, we associate each country with a count of 1 for each amateur observatory.

Mapper 1

```
protected void map(LongWritable key, // Input key type
Text value, // Input value type
Context context) throws IOException, InterruptedException {

    // Extract the fields from the record
    // ObservatoryID, Name, Lat, Lon, Country, Continent, Amateur
    String[] fields = value.toString().split(",");
    String country = fields[4];
    String flag = fields[6];

    // Only process amateur observatories
    if(flag.equals("True"))
        context.write(new Text(country), new IntWritable(1));
}
```

In the Reducer of Job 1, we compute the local maximum for optimization purposes.

Reducer 1

```

private int maxCount;
private String maxCountry;

@Override
protected void setup(Context context) throws IOException,
    InterruptedException {
    maxCount = -1;
    maxCountry = null;
}

@Override
protected void reduce(Text key, // Input key type
    Iterable<IntWritable> values, // Input value type
    Context context) throws IOException,
    InterruptedException {

    String country = key.toString();
    int count = 0;

    // Sum the counts for each country
    for(IntWritable v : values)
        count += v.get();

    // Track the country with the highest count
    if ((count > maxCount) || (count == maxCount &&
        country.compareTo(maxCountry) < 0)) {
        maxCount = count;
        maxCountry = country;
    }
}

@Override
protected void cleanup(Context context) throws
    IOException, InterruptedException {
    // Emit the local maximum for the reducer
    if (maxCount != -1) {
        context.write(new Text(maxCountry), new
            IntWritable(maxCount));
    }
}

```

If the key sent to the reducer is of type `IntWritable` instead of `Text`, the computation of the local maximum will differ slightly, as the comparison will be based on numeric values rather than string values

Reducer 1 - version 2

```

private int maxCount;
private int maxYear;

@Override
protected void setup(Context context) throws IOException,
    InterruptedException {
    maxCount = -1;
    maxYear = Integer.MAX_VALUE;
}

@Override
protected void reduce(IntWritable key, // Input key type
    Iterable<IntWritable> values, // Input value
    type
    Context context) throws IOException,
    InterruptedException {

    int numOccurrences = 0;
    int currentYear = key.get();

    // Compute the number of occurrences of the current year
    for (IntWritable value : values) {
        numOccurrences = numOccurrences + value.get();
    }

    // Check if this is the local maximum
    if (maxCount < numOccurrences || (maxCount == numOccurrences
        && currentYear < maxYear)) {
        maxCount = numOccurrences;
        maxYear = currentYear;
    }
}

@Override
protected void cleanup(Context context) throws IOException,
    InterruptedException {
    // Emit the local maximum and the associated year
    if (maxCount != -1)
        context.write(new IntWritable(maxCount), new
            IntWritable(maxYear));
}

```

In Mapper 2, we use `NullWritable.get()` as the key to focus only on the maximum value, as we don't need the key anymore.

Mapper 2

```
protected void map(
    Text key,          // Input key type
    Text value,        // Input value type
    Context context) throws IOException,
    InterruptedException {

    String country = key.toString();
    String count = value.toString();

    // Emit the country and its corresponding count with
    // NullWritable as the key
    context.write(NullWritable.get(), new Text(country +
        "_" + count));
}
```

Finally, in Reducer 2, we compute the global maximum from the values emitted in Mapper 2.

Reducer 2

```
// Compute global maximum
@Override
protected void reduce(
    NullWritable key, // Input key type
    Iterable<Text> values, // Input value type
    Context context) throws IOException,
    InterruptedException {

    int maxCount = -1;
    String maxCountry = null;

    // Iterate over all the values and find the country
    // with the highest count
    for(Text v : values) {
        String data = v.toString();
        String[] fields = data.split("_");
        String country = fields[0];
        Integer count = Integer.parseInt(fields[1]);

        // Update max count and country based on the
        // comparison
        if(count > maxCount || (maxCount == count &&
            country.compareTo(maxCountry) < 0)) {
            maxCount = count;
            maxCountry = country;
        }
    }

    // Emit the final result: Country and the number of
    // amateur observatories
    context.write(new Text(maxCountry), new IntWritable(
        maxCount));
}
```

This solution computes the country with the greatest number of amateur observatories, storing the result in the output HDFS folder. If there are multiple countries with the same maximum, it selects the one with the lexicographically smallest name.

For the same reason explained earlier, the reducer in the second job will need to be adjusted accordingly to handle the different key type

Reducer 2 - Version 2

```

protected void reduce(NullWritable key, // Input key type
    Iterable<Text> values, // Input value type
    Context context) throws IOException,
    InterruptedException {

    int maxCount = -1;
    int maxYear = Integer.MAX_VALUE;

    int currentYear;
    int numOccurrences;

    for (Text val : values) {
        String[] fields = val.toString().split("_");
        currentYear = Integer.parseInt(fields[0]);
        numOccurrences = Integer.parseInt(fields[1]);

        // Check if this is the global maximum
        if (maxCount < numOccurrences || (maxCount ==
            numOccurrences && currentYear < maxYear)) {
            maxCount = numOccurrences;
            maxYear = currentYear;
        }
    }
    // Store the result
    context.write(new IntWritable(maxYear), new
        IntWritable(maxCount));
}

```

7 Spark Exam

These are some tips to solve the Spark exercises based on the passed exam

7.1 keys() Function

In Apache Spark, the `keys()` function is used to extract the keys from an RDD of key-value pairs. When you apply `keys()` to an RDD of tuples (or pairs), it returns an RDD containing just the keys from those tuples.

7.2 values() Function

In Apache Spark, the `values()` function is used to extract the values from an RDD of key-value pairs. When you apply `values()` to an RDD of tuples (or pairs), it returns an RDD containing just the values from those tuples.

7.3 Subtracting Elements in a Pair RDD

In Spark, the `subtract()` method operates on entire elements, meaning that if you apply it to a pair RDD (an RDD of key-value pairs), it will only remove exact key-value matches. If you need to subtract elements based on either keys or values, you must first transform the RDD accordingly before using `subtract()`.

7.3.1 Subtracting Based on Keys

To remove all pairs that have the same key in another RDD, you can extract the keys using `keys()` and filter the original RDD:

```
rdd1 = sc.parallelize([(1, "a"), (2, "b"), (3, "c")])
rdd2 = sc.parallelize([(2, "x"), (3, "y")])

# Extract keys and subtract
keys_to_remove = rdd2.keys()
filtered_rdd = rdd1.filter(lambda pair: pair[0] not in
                           keys_to_remove.collect())

print(filtered_rdd.collect()) # Output: [(1, "a")]
```

7.3.2 Subtracting Based on Values

To remove elements that have the same value but keep other keys, you can extract values and use `subtract()` as follows:

```

rdd1 = sc.parallelize([(1, "a"), (2, "b"), (3, "c")])
rdd2 = sc.parallelize([(4, "b"), (5, "c")])

# Extract values and subtract
values_to_remove = rdd2.values()
filtered_rdd = rdd1.filter(lambda pair: pair[1] not in
                           values_to_remove.collect())

print(filtered_rdd.collect()) # Output: [(1, "a")]

```

7.3.3 Key-Value Pair Subtraction (Exact Matches)

If you want to subtract exact key-value pairs, you can use `subtract()` directly:

```

rdd1 = sc.parallelize([(1, "a"), (2, "b"), (3, "c")])
rdd2 = sc.parallelize([(2, "b")])

result_rdd = rdd1.subtract(rdd2)

print(result_rdd.collect()) # Output: [(1, "a"), (3, "c")]

```

This approach ensures that only exact key-value matches are removed while keeping other elements intact.

7.4 Usage of `reduceByKey`

`reduceByKey` is a transformation in PySpark that merges values for the same key using an associative and commutative function. It is commonly used for aggregation tasks such as summing, counting, or finding maximum/minimum values.

Basic Syntax

```
rdd.reduceByKey(lambda v1, v2: operation)
```

- `v1` and `v2` are two values associated with the same key.
- The lambda function defines how to combine these values.
- The result is a new RDD with the same keys but reduced values.

Summing Values

If the RDD consists of key-value pairs (`key`, `int`), you can sum all values for each key:

```

rdd = sc.parallelize([("A", 1), ("A", 2), ("B", 3), ("B", 4)
    ])
result = rdd.reduceByKey(lambda v1, v2: v1 + v2)
print(result.collect())
# Output: [('A', 3), ('B', 7)]

```

Finding the Maximum Value

To get the maximum value per key:

```

rdd = sc.parallelize([("A", 3), ("A", 5), ("B", 2), ("B", 8)
    ])
result = rdd.reduceByKey(lambda v1, v2: max(v1, v2))
print(result.collect())
# Output: [('A', 5), ('B', 8)]

```

Counting Occurrences

If each key appears multiple times and you want to count occurrences:

```

rdd = sc.parallelize(["apple", "banana", "apple", "orange", "
    banana", "apple"])
mapped_rdd = rdd.map(lambda fruit: (fruit, 1))
result = mapped_rdd.reduceByKey(lambda v1, v2: v1 + v2)
print(result.collect())
# Output: [('apple', 3), ('banana', 2), ('orange', 1)]

```

Using Tuples as Values

If each key has a tuple (*val1*, *val2*), you can reduce by summing element-wise:

```

rdd = sc.parallelize([("A", (3, 1)), ("A", (2, 4)), ("B", (5, 2)), ("B", (1, 6))])
result = rdd.reduceByKey(lambda v1, v2: (v1[0] + v2[0], v1[1] + v2[1]))
print(result.collect())
# Output: [('A', (5, 5)), ('B', (6, 8))]
# Equivalent
rdd = sc.parallelize([("A", (3, 1)), ("A", (2, 4)), ("B", (5, 2)), ("B", (1, 6))])
rdd = rdd.map(lambda p: p[1]) # Ora l'RDD contiene solo le tuple (dim1, dim2)
result = rdd.reduce(lambda v1, v2: (v1[0] + v2[0], v1[1] + v2[1]))
print(result)
# Output: (11, 13)

```

Using Tuples as Values with Max Operation

If each key has a tuple (*val1*, *val2*), you can reduce by computing the element-wise maximum:

```

rdd = sc.parallelize([("A", (3, 1)), ("A", (2, 4)), ("B", (5, 2)), ("B", (1, 6))])
result = rdd.reduceByKey(lambda v1, v2: (max(v1[0], v2[0]), max(v1[1], v2[1])))
print(result.collect())
# Output: [('A', (3, 4)), ('B', (5, 6))]

```

In this example:

- The input RDD consists of key-value pairs where the values are tuples.
- The function used in 'reduceByKey' applies 'max' to each element of the tuple.
- The result contains the maximum value for each tuple position within each key group.

This approach can be extended to tuples with more elements by applying 'max' to each index accordingly.

7.5 When to Use reduceByKey and reduce

In Apache Spark, both `reduce` and `reduceByKey` are used for aggregating data, but they differ in how they operate and when you should use them. This guide explains the difference and when to use each function.

When to Use reduceByKey

`reduceByKey` is used when working with key-value pairs, where each key is associated with multiple values, and you need to aggregate the values for each key. It performs the aggregation in a distributed manner, which helps optimize performance by reducing the amount of data shuffled between nodes.

Use `reduceByKey` when:

- You are working with an RDD of key-value pairs (e.g., (`key`, `value`)).
- You want to aggregate values for each key (e.g., sum, average, etc.).
- You need an optimized, distributed reduction process.

Example

```
rdd = sc.parallelize([("A", 1), ("A", 2), ("B", 3)])
result = rdd.reduceByKey(lambda x, y: x + y)
print(result.collect()) # Output: [('A', 3), ('B', 3)]
```

When to Use reduce

`reduce` is used for aggregating all elements in an RDD into a single value. It does not consider keys, so it is best used when you want to reduce the entire dataset without any regard to key-value pairs.

Use `reduce` when:

- You have an RDD of values (e.g., (`value`)), without any associated keys.
- You want to combine all elements in the RDD into a single result (e.g., sum of all values, maximum, etc.).
- You do not need key-based aggregation.

Example

```
rdd = sc.parallelize([1, 2, 3, 4])
result = rdd.reduce(lambda x, y: x + y)
print(result) # Output: 10
```

7.6 Using the mapValues Function

The `mapValues` function in Spark is used to transform only the values of key-value pairs in an RDD while keeping the keys unchanged.

```

rdd = sc.parallelize([("Laptop", 1000), ("Phone", 500), ("
    Tablet", 300)])

# Apply a 10% discount on sales numbers
discounted_rdd = rdd.mapValues(lambda sales: sales * 0.9)

print(discounted_rdd.collect())
# Output: [('Laptop', 900.0), ('Phone', 450.0), ('Tablet',
    270.0)]

```

In this example:

- The input RDD consists of key-value pairs where the values represent sales numbers.
- The function used in ‘mapValues’ applies a 10% discount to each value.
- The keys (e.g., “Laptop”, “Phone”, “Tablet”) remain unchanged.

This function is particularly useful when working with key-value RDDs where only the values need modification.

7.7 Using the subtractByKey Function

The `subtractByKey` operation returns an RDD that contains the elements from the first RDD where the keys do not exist in the second RDD.

```

# Example RDDs
rdd1 = sc.parallelize([("apple", 3), ("banana", 5), ("orange",
    2), ("grape", 4)])
rdd2 = sc.parallelize([("banana", 20), ("grape", 30)])

# Subtract keys in rdd2 from rdd1
result_rdd = rdd1.subtractByKey(rdd2)

# Output: [('apple', 3), ('orange', 2)]
print(result_rdd.collect())

```

The result excludes any key-value pairs in `rdd1` where the keys exist in `rdd2`.

7.8 Accessing Fields in Nested Tuples - Single pair

If you have a tuple structured as follows:

```
(JobID, ((Salary, OfferId)))
```

you can access individual fields using **indexing** in Spark.

```

rdd = sc.parallelize([
    (1, ((50000, "01"))),
    (2, ((60000, "02"))),
    (3, ((70000, "03")))
])

# Extract individual fields
def extract_fields(record):
    job_id = record[0] # JobID
    salary = record[1][0] # Salary
    offer_id = record[1][1] # OfferId
    return (job_id, salary, offer_id)

# Map the RDD with the function
parsed_rdd = rdd.map(extract_fields)
# Extract to the pair (Salary, OfferId)
rdd2 = rdd.map(lambda p: p[1])

```

7.9 Accessing Fields in Nested Tuples - Double pair

If you have a tuple structured as follows:

(JobID, ((Salary, OfferId), (Country, Title)))

you can access individual fields using **indexing** in Spark.

```

rdd = sc.parallelize([
    (1, ((50000, "01"), ("USA", "Engineer"))),
    (2, ((60000, "02"), ("UK", "Manager"))),
    (3, ((70000, "03"), ("Canada", "Analyst")))
])

# Extract individual fields
def extract_fields(record):
    job_id = record[0] # JobID
    salary = record[1][0][0] # Salary
    offer_id = record[1][0][1] # OfferId
    country = record[1][1][0] # Country
    title = record[1][1][1] # Title
    return (job_id, salary, offer_id, country, title)

# Map the RDD with the function
parsed_rdd = rdd.map(extract_fields)
# This gives you (Salary, OfferId) and (Country, Title)
rdd2 = rdd.map(lambda p: (p[1][0], p[1][1]))

```

7.10 How the join works in PySpark

The `join` method in PySpark is used to combine two RDDs based on a common key. The result of the `join` is a new RDD in which the matching keys from both RDDs are merged together.

Structure of join

The `join` is applied on two **key-value pair RDDs**, so `RDD1.join(RDD2)` produces a new RDD with the structure:

$$(key, (value1, value2))$$

Where:

- `key` is the value on which the match occurs.
- `value1` comes from RDD1.
- `value2` comes from RDD2.

Practical Example

Imagine you have two RDDs:

```
RDD1 = sc.parallelize([
    (1, "Alice"),
    (2, "Bob"),
    (3, "Charlie")
])

RDD2 = sc.parallelize([
    (1, "Data_Scientist"),
    (2, "Engineer"),
    (3, "Designer"),
    (4, "Manager") # No match in RDD1
])
```

If you perform:

```
result = RDD1.join(RDD2)
result.collect()
```

You will get:

```
[(1, ("Alice", "Data Scientist")), (2, ("Bob", "Engineer")), (3, ("Charlie", "Designer"))]
```

Note: The key 4 from RDD2 does not appear in the result because there is no matching value in RDD1.

What is join useful for?

1. Linking different datasets with a common key:

- Example: Join student data with exam scores based on the `student ID`.

2. Filtering relevant information:

- Example: If you have a dataset with job offers and another with signed contracts, you can join them to find only the offers that led to a contract.

3. Enriching datasets:

- Example: Suppose you have a dataset with transactions (`customer ID`, `amount`) and another with customer information (`customer ID`, `name`). A `join` can add the customer name to each transaction.

None placeholder

When performing `join` operations between two RDDs in PySpark, it is important that the data structures are compatible. In the case of a `join`, PySpark expects the keys to be aligned and for the values associated with the keys to be similar. For example, if one of the RDDs has a different format (e.g., an RDD with a pair of values while the other has only one), the join operation will not work as expected.

Imagine you have two RDDs with different structures:

Example

```
RDD1 = sc.parallelize([
    (1, "Alice"),
    (2, "Bob"),
    (3, "Charlie")
])

RDD2 = sc.parallelize([
    (1, "Data_Scientist", "USA"),
    (2, "Engineer", "Canada"),
    (3, "Designer", "UK")
])
```

Here, RDD1 has pairs of (key, name), while RDD2 has pairs of (key, name, country). If you try to perform the join without aligning the structures, PySpark will not be able to merge the data correctly. To align the structures, you can add a placeholder (`None`) in the first RDD to "complete" the pair and make sure both have the same number of values:

```

RDD1 = RDD1.map(lambda x: (x[0], (x[1], None))) # Add None
      as a placeholder
result = RDD1.join(RDD2)
result.collect()

```

Now, the structure of RDD1 becomes:

```

[(1, ("Alice", None)),
 (2, ("Bob", None)),
 (3, ("Charlie", None))]

```

And when you perform the join, the result will be:

```

[(1, (("Alice", None), "Data_Scientist", "USA")),
 (2, (("Bob", None), "Engineer", "Canada")),
 (3, (("Charlie", None), "Designer", "UK"))]

```

In this way, the `join` works correctly even though the two RDDs originally had different structures. `None` serves to "complete" the data in the first RDD so that both structures have the same number of values for each key, allowing the `join` to combine the data correctly.

If you hadn't added `None`, PySpark might not have been able to perform the join as expected, especially if there were differences in the tuple sizes.

7.11 Types of Joins in Spark

Inner Join

An **inner join** returns only the records that have matching keys in both datasets. If there is no match, the record is excluded from the result.

```

# Sample RDDs of key-value pairs
rdd1 = sc.parallelize([("apple", 3), ("banana", 5), ("orange", 2)])
rdd2 = sc.parallelize([("apple", 10), ("banana", 20), ("grape", 30)])

# Perform an inner join
joined_rdd = rdd1.join(rdd2)

# Output: [('apple', (3, 10)), ('banana', (5, 20))]
print(joined_rdd.collect())

```

Left Outer Join

A **left outer join** returns all records from the left dataset, along with the matching records from the right dataset. If there is no match in the right dataset, the result will contain `None` for the right dataset.

```
# Perform a left outer join
left_outer_joined_rdd = rdd1.leftOuterJoin(rdd2)

# Output: [('apple', (3, 10)), ('banana', (5, 20)), ('orange', (2, None))]
print(left_outer_joined_rdd.collect())
```

Right Outer Join

A **right outer join** returns all records from the right dataset, along with the matching records from the left dataset. If there is no match in the left dataset, the result will contain `None` for the left dataset.

```
# Perform a right outer join
right_outer_joined_rdd = rdd1.rightOuterJoin(rdd2)

# Output: [('apple', (3, 10)), ('banana', (5, 20)), ('grape', (None, 30))]
print(right_outer_joined_rdd.collect())
```

7.12 Comparing Total Statistics Across Two Different Years

When computing total statistics for two different years and subsequently comparing them, a common approach involves the following pattern:

- **Filtering:** Select only the records corresponding to the years of interest (e.g., 2019 and 2021).
- **Mapping:** Extract the relevant product identifier and sales data, encoding the sales for each year separately.
- **Reducing:** Aggregate the total sales per product for each year.

The following Spark-based implementation follows this pattern:

```
# Filter function to retain only records from the years 2019
and 2021
def filterByYear(line):
    fields = line.split(",")
    date = fields[1]
    return date.startswith("2019") or date.startswith("2021")

# Mapping function: extract ProductID and sales, separating
values for each year
def mapProductSales(line):
    fields = line.split(",")
    product_id = fields[0]
    date = fields[1]
    num_sales = int(fields[2])

    if date.startswith("2019"):
        return (product_id, (num_sales, 0)) # (sales_2019,
            sales_2021)
    else:
        return (product_id, (0, num_sales)) # (sales_2019,
            sales_2021)
```

```
# Apply the transformations
salesPerYearRdd = (salesRdd
    .filter(filterByYear) # Keep only 2019
    and 2021 sales
    .map(mapProductSales) # Map to (
    ProductID, (sales_2019, sales_2021))
    .reduceByKey(lambda v1, v2: (v1[0] + v2[0], v1[1] + v2
    [1])) # Sum sales per year
)
```

After applying these transformations, the final structure of `salesPerYearRdd` is:

```
RDD[(ProductID: str, (TotalSales2019: int, TotalSales2021: int))] (1)
```

For example, the resulting RDD might contain:

```
[
    ("ID1", (5000, 4500)), # ID1 sold 5000 units in 2019,
                        4500 in 2021
    ("ID2", (12000, 13000)), # ID2 sold more in 2021
    ("ID3", (800, 600)), # ID3 sales dropped from 800 to 600
]
```

This structure allows for an efficient comparison of sales trends across the two years.

If the number of sales is not explicitly available in the database but needs to be computed by counting the number of records corresponding to each year, we can take a different approach. Instead of directly extracting the sales data, we can count the occurrences of each record that corresponds to a particular year. The following shows how to implement this approach:

```
def userIdCounters(line):
    fields = line.split(",")
    userId = fields[1]
    year = fields[0].split("/") [0]

    # Increment count for the corresponding year
    if(year == "2022"):
        return (userId, (1, 0)) # Add 1 to count for 2022, 0
                                # for 2023
    else:
        return (userId, (0, 1)) # Add 1 to count for 2023, 0
                                # for 2022
```

Next, we apply the transformations to count the occurrences for each user in both 2022 and 2023:

```
# Apply the transformations: count the occurrences for each
# user in both 2022 and 2023
userCountPurchases2223 = purchases2223.map(userIdCounters) \
    .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \
    .cache() # Cache the result for further use
```

Finally, we compute the maximum number of purchases for each year (2022 and 2023):

```
# Find the maximum number of purchases for each year (2022
  and 2023)
maxPurchases2223 = userCountPurchases2223.values() \
    .reduce(lambda a, b: (max(a[0], b[0]), max(a[1], b[1])))
    # Find max for 2022 and 2023
```

In this approach:

- The `userIdCounters` function counts the number of records for each user in both 2022 and 2023. We use tuples (1, 0) for 2022 and (0, 1) for 2023, respectively.
- The `reduceByKey` operation sums these counts for each user.
- Finally, the `reduce` operation computes the maximum number of occurrences (purchases) for each year, giving the highest total number of purchases among users for both years.

This technique allows you to analyze the frequency of events (such as purchases) per user for each year and compare them efficiently.

7.13 Computation of Max, Min, Total, and Count for a Specific Field

In this type of exercise, we are tasked with computing statistics for a specific field in a dataset, grouped by a unique identifier (in this example, the field is `duration`). The goal is to compute the following statistics for each user:

- The average duration of the meetings organized by the user.
- The maximum duration of the meetings organized by the user.
- The minimum duration of the meetings organized by the user.
- The total number of meetings organized by the user.

The statistics should only consider users with a `Business` pricing plan, as specified in the exercise description this is the reason why there is also a join performed.

```
def mapValStat(t):
    duration = t[0]
    # Return a tuple (min_duration, max_duration,
    total_duration, num_meetings)
    return (duration, duration, duration, 1)
```

We use `mapValues` to transform the values into a tuple containing the initial statistics for each meeting. Finally, `reduceByKey` is used to compute the desired statistics (min, max, total, count) for each user:

```
meetingStats = durationRDD.join(businessUsers)\
    .mapValues(mapValStat)\
    .reduceByKey(lambda t1, t2: (
        t1[0] if t1[0] < t2[0] else t2[0],\
        t1[1] if t1[1] > t2[1] else t2[1],\
        t1[2] + t2[2],\
        t1[3] + t2[3]))
```

In the resulting `meetingStats`, the data is structured as an RDD of key-value pairs:

- **Key:** The user identifier (UID), which associates the meeting statistics with a specific user.
- **Value:** A tuple of four integers: (`min_duration`, `max_duration`, `total_duration`, `num_meetings`).

For example, a key-value pair might look like this:

```
("User1000", (60, 120, 270, 3))
```

To compute the average meeting duration and format the results in the required order (average, max, min), we use the following transformation:

Computing Average Duration

```
res1 = meetingStats.mapValues(lambda t: (t[2] / t[3], t[1], t
    [0])) # (avg, max, min)
```

Finally, the computed statistics are stored in an HDFS output folder as follows:

Storing Results in Output

```
res1.saveAsTextFile(outputPath1)
```

7.14 Union Function for RDD and Pair RDD in Apache Spark

In Apache Spark, the `union` function combines two RDDs into a new one. It behaves slightly differently for standard RDDs and Pair RDDs. Here's how it works for each:

Union for Standard RDDs

For standard RDDs (i.e., RDDs containing basic data types like integers, strings, etc.), the `union` function combines two RDDs by appending the elements of one to the other.

Example

```

from pyspark import SparkContext

sc = SparkContext("local", "Union_Example")

rdd1 = sc.parallelize([1, 2, 3])
rdd2 = sc.parallelize([4, 5, 6])

result_rdd = rdd1.union(rdd2)
print(result_rdd.collect()) # Output: [1, 2, 3, 4, 5, 6]

```

- **Input:** Two RDDs (rdd1 and rdd2) containing basic data types.
- **Output:** A new RDD that contains all the elements from both RDDs, **including duplicates** if any.

Union for Pair RDDs

A *Pair RDD* is an RDD of key-value pairs, where each element is a tuple (**key**, **value**). The `union` function for Pair RDDs behaves the same way as for standard RDDs in terms of combining the data, but it maintains the key-value structure. The operation does not perform any grouping or aggregation of keys.

Example

```

rdd1 = sc.parallelize([("a", 1), ("b", 2)])
rdd2 = sc.parallelize([("c", 3), ("d", 4)])

result_pair_rdd = rdd1.union(rdd2)
print(result_pair_rdd.collect()) # Output: [('a', 1), ('b', 2), ('c', 3), ('d', 4)]

```

- **Input:** Two Pair RDDs (rdd1 and rdd2) containing key-value pairs.
- **Output:** A new Pair RDD that contains all the key-value pairs from both input RDDs, **including duplicates**.

Key Notes for Pair RDDs

- The union operation **does not aggregate or combine values** for identical keys. It simply appends the key-value pairs.
- If both Pair RDDs have the same keys, those keys will appear in the result with their corresponding values, which could lead to duplicates.

Example with Duplicate Keys

```
rdd1 = sc.parallelize([("a", 1), ("b", 2)])
rdd2 = sc.parallelize([("a", 3), ("b", 4)])

result_pair_rdd = rdd1.union(rdd2)
print(result_pair_rdd.collect()) # Output: [('a', 1), ('b', 2), ('a', 3), ('b', 4)]
```

- **Input:** Two Pair RDDs with overlapping keys ("a" and "b").
- **Output:** The keys "a" and "b" appear twice, once from each RDD.

Removing Duplicates After Union

If you want to remove duplicates after performing a `union` operation, you can apply `distinct()`:

```
rdd1.union(rdd2).distinct()
```

For Pair RDDs, this will remove duplicate key-value pairs.

Example with `distinct()`:

```
rdd1 = sc.parallelize([("a", 1), ("b", 2)])
rdd2 = sc.parallelize([("a", 3), ("b", 2)])

result_pair_rdd = rdd1.union(rdd2).distinct()
print(result_pair_rdd.collect()) # Output: [('a', 1), ('b', 2), ('a', 3)]
```